



Faculdades de Campinas

ENGENHARIA DA COMPUTAÇÃO

Tratamento e Armazenamento de Informação

Atividade 3.3 - Questões Dissertativas:

Projeto SQL e NoSQL

GRUPO:

Felipe Furlan RA 202210339

Henrique Biruel RA 202210347

Henrique Palorca RA 202210348

Campinas
Setembro de 2026

1 INTRODUÇÃO

O presente trabalho apresenta as respostas às dez questões dissertativas propostas para o projeto SQL e NoSQL da disciplina Tratamento e Armazenamento de Informação. As questões comparam as duas versões do e-commerce desenvolvido ao longo da disciplina — a primeira sobre um banco relacional PostgreSQL, a segunda sobre um banco de documentos Apache CouchDB — e as estendem a cenários de maior porte: expansão do catálogo, picos de acesso, relatórios comerciais, implantação em nuvem e tolerância a falhas (MARCUSO, 2026). A discussão toma como referência concreta a loja Torra & Terra, construída pelo grupo nas duas versões (FURLAN; BIRUEL; PALORCA, 2026), e adota como fio condutor a ideia de que não existe modelo superior em abstrato: cada escolha desloca custos entre o banco, a aplicação e a operação, e precisa ser justificada pelo padrão de acesso e pelos requisitos do negócio.

2 QUESTÃO 1 — MODELAGEM DOS DADOS: NORMALIZAÇÃO E AGREGAÇÃO

No projeto SQL, clientes, produtos, pedidos e itens são normalmente distribuídos em tabelas relacionadas; no projeto NoSQL, parte dessas informações pode ser agregada em documentos. Em quais situações a normalização do SQL é mais vantajosa e em quais situações a agregação de dados do NoSQL simplifica a aplicação?

A normalização registra cada fato em um único lugar e relaciona as tabelas por chaves, o que elimina redundância e anomalias de atualização e permite que o próprio banco garanta a integridade referencial (ELMASRI; NAVATHE, 2011). Ela é mais vantajosa quando os dados são compartilhados e alterados por vários fluxos — o cadastro de um cliente usado por pedidos, entregas e cobrança —, quando as relações são de muitos para muitos e quando o negócio precisa de consultas imprevistas que cruzam entidades, como relatórios gerenciais. Nesses casos, cada informação é atualizada uma única vez, e o otimizador compõe as junções no momento da consulta.

A agregação simplifica a aplicação quando os dados são lidos e gravados juntos, formando uma unidade natural de acesso (SADALAGE; FOWLER, 2012). No projeto NoSQL do grupo, o pedido passou a ser um único documento com os itens

embutidos, cada um com nome, moagem, quantidade e preço do momento da compra: o histórico do cliente é lido pedido a pedido, sem junção. No projeto SQL, a mesma tela dependia das tabelas pedidos e itens_pedido, ligadas por chave estrangeira (FURLAN; BIRUEL; PALORCA, 2026). A agregação favorece ainda atributos variáveis e a evolução do formato sem migração. O limite aparece quando o dado embutido tem vida própria: produto e cliente continuaram como documentos separados e referenciados, porque mudam independentemente do pedido e são compartilhados por muitos deles.

3 QUESTÃO 2 — CONSISTÊNCIA VERSUS FLEXIBILIDADE

Imagine que um cliente finalize uma compra enquanto o estoque está sendo atualizado simultaneamente. Como o projeto SQL e o projeto NoSQL tratariam essa situação? Qual seria o impacto das diferentes estratégias de consistência na experiência do cliente e na operação da empresa?

No projeto SQL, o checkout é uma transação: pedido, itens e baixa de estoque são gravados entre BEGIN e COMMIT, e a leitura do estoque usa SELECT ... FOR UPDATE, que bloqueia as linhas dos produtos até o fim da transação. Uma segunda compra do mesmo produto espera a primeira terminar e só então enxerga o valor atualizado; se o estoque não bastar, a transação inteira é desfeita, e a restrição CHECK (estoque $\geq$ 0) atua como última barreira no próprio banco. No projeto NoSQL não há transação entre documentos: o CouchDB aplica controle otimista, e uma gravação feita sobre uma revisão (_rev) desatualizada recebe HTTP 409, obrigando a aplicação a reler o café e reavaliar o estoque. Como pedido e cafés precisam mudar juntos, o grupo implementou o checkout como uma saga — pedido PENDENTE, reserva do estoque, confirmação e, em caso de falha, compensação que cancela o pedido e devolve apenas o que foi reservado (GARCIA-MOLINA; SALEM, 1987; FURLAN; BIRUEL; PALORCA, 2026).

As duas estratégias impedem vender o mesmo lote duas vezes, mas aparecem de forma diferente. Para o cliente, o SQL se manifesta como uma espera curta sob disputa; o NoSQL, como novas tentativas transparentes ou, no pior caso, uma compra recusada com mensagem que indica o café que faltou. Para a empresa, o modelo relacional concentra a garantia no banco, com o risco de filas de

bloqueio e impasses em picos de acesso, enquanto o modelo de documentos transfere para o código e para a operação a responsabilidade por pedidos pendentes, compensações e rotinas de reconciliação — mais liberdade de distribuição, com mais cenários a testar e a monitorar (KLEPPMANN, 2017).

4 QUESTÃO 3 — EVOLUÇÃO DO CATÁLOGO

Suponha que o e-commerce passe a vender eletrônicos, roupas, alimentos e serviços digitais, todos com atributos muito diferentes. Como essa expansão afetaria o modelo de dados SQL? Como afetaria um banco NoSQL baseado em documentos? Que consequências haveria para manutenção e governança?

No modelo relacional, atributos heterogêneos pressionam o esquema. As alternativas clássicas têm custos conhecidos: uma tabela única com muitas colunas opcionais, quase sempre nulas; uma tabela por família de produto, que multiplica junções e migrações; ou o modelo entidade-atributo-valor, flexível, mas difícil de consultar e de validar (ELMASRI; NAVATHE, 2011). O PostgreSQL oferece ainda um caminho híbrido, com os atributos comuns em colunas e os específicos em uma coluna JSONB indexável. Em todos os casos, cada nova categoria tende a exigir migração de esquema e implantação coordenada com a aplicação. Em um banco de documentos, cada produto carrega os próprios atributos: um notebook descreve processador e memória, uma camiseta descreve tamanho e tecido, e ambos convivem no mesmo banco sem alteração de esquema (SADALAGE; FOWLER, 2012).

A flexibilidade, porém, não elimina as regras; apenas muda o lugar onde elas vivem. No SQL, o esquema é o próprio DDL, visível, auditável e imposto a qualquer cliente do banco. No modelo de documentos, ele passa a morar no código, em funções de validação e na documentação: no projeto NoSQL do grupo, a `validate_doc_update` aplica no CouchDB as regras que no relacional eram CHECK, e todo documento registra `versao_esquema` para que formatos antigos e novos convivam (FURLAN; BIRUEL; PALORCA, 2026). A governança passa a exigir dicionário de atributos por categoria, responsáveis definidos e índices criados apenas para as consultas previstas; sem essa disciplina, a flexibilidade se converte

em documentos inconsistentes e em relatórios que precisam tratar dezenas de variações do mesmo campo.

5 QUESTÃO 4 — ESCALABILIDADE EM UM PICO DE DEMANDA

Durante uma campanha semelhante à Black Friday, o volume de consultas e pedidos aumenta dez vezes. Compare como os projetos SQL e NoSQL poderiam ser escalados. Analise escala vertical, réplicas, particionamento, sharding e distribuição horizontal, considerando complexidade e custo.

O caminho mais simples para o projeto SQL é a escala vertical: uma instância com mais CPU, memória e disco mais rápido, sem mudança de código, mas com teto físico, custo que cresce mais rápido que a capacidade e, em geral, uma janela de indisponibilidade na troca. Em seguida vêm as réplicas de leitura, que absorvem catálogo e consultas enquanto o primário concentra as escritas, ao custo do atraso de replicação. O particionamento de tabelas por data mantém índices e manutenção sob controle, mas continua em um único servidor; distribuir escritas exige sharding, por extensões ou pela própria aplicação, o que torna caras as junções e as transações entre shards (KLEPPMANN, 2017). Bancos NoSQL distribuídos, como Cassandra e DynamoDB, foram projetados para crescer horizontalmente: os dados são repartidos por uma chave de partição, e novos nós absorvem carga com rebalanceamento automático (DECANDIA et al., 2007). O desafio desloca-se para a modelagem — uma chave concentradora cria hotspots justamente no pico — e para a consistência entre réplicas. O Quadro 1 resume as estratégias.

Quadro 1 — Estratégias de escala para um pico de dez vezes no volume

Estratégia	Projeto SQL (PostgreSQL)	Projeto NoSQL (documentos)	Complexidade e custo
Escala vertical	Primeira opção; nenhuma mudança de código	Possível, mas desperdiça o modelo distribuído	Baixa complexidade; custo alto e teto físico
Réplicas	Leitura nas réplicas, escrita no primário	Réplicas nativas, com leitura e escrita por quórum	Média; atraso e consistência eventual nas leituras
Particionamento	Por intervalo, como pedidos por mês, em um servidor	Por chave de partição, distribuído entre nós	Média; depende de conhecer o padrão de acesso
Sharding	Por extensão ou pela aplicação; junções entre shards caras	Nativo, com rebalanceamento automático	Alta no SQL; risco de hotspot no NoSQL

Estratégia	Projeto SQL (PostgreSQL)	Projeto NoSQL (documentos)	Complexidade e custo
Cache e fila	Cache do catálogo e fila de pedidos aliviam o banco	Mesmo efeito	Baixa a média; adia a necessidade de escalar o banco

Fonte: elaborado pelos autores com base em Kleppmann (2017) e em DeCandia et al. (2007).

Para um pico previsível e temporário, cache no catálogo, réplicas de leitura, fila para suavizar a entrada de pedidos e capacidade elástica de serviços gerenciados costumam resolver o problema com menos risco do que um redesenho para sharding. O NoSQL distribuído compensa quando o crescimento de escrita é permanente e supera a capacidade de um único servidor. O CouchDB do projeto do grupo, por exemplo, roda hoje em um único nó; em cluster, repartiria o banco em shards e réplicas, mas passaria a conviver com conflitos de revisão que o nó único não produz (FURLAN; BIRUEL; PALORCA, 2026).

6 QUESTÃO 5 — CONSULTAS E RELATÓRIOS

O departamento comercial deseja responder perguntas como: “quais clientes de Campinas compraram notebooks nos últimos seis meses e gastaram mais de R\$ 5.000?”. Compare a dificuldade e a eficiência de realizar esse tipo de consulta no projeto SQL e no projeto NoSQL. Em que medida joins, agregações e desnormalização interferem nessa decisão?

No projeto SQL, a pergunta vira uma consulta declarativa direta: junção de clientes, pedidos, itens e produtos, filtro por cidade, categoria e período, agrupamento por cliente e condição HAVING sobre a soma dos valores. O otimizador escolhe a ordem das junções e usa índices sobre a data do pedido, a categoria e o cliente; com volume moderado, a resposta é eficiente, e a consulta pode ser reescrita livremente para a próxima pergunta do comercial (ELMASRI; NAVATHE, 2011). Vale notar que nenhuma das duas versões do projeto guarda hoje a cidade do cliente, atributo que teria de ser incluído em ambas antes do relatório.

No projeto NoSQL, a mesma pergunta depende de como os dados foram agregados. Sem junções, cada condição precisa estar disponível no documento consultado: se o pedido guardar a cidade do cliente e a categoria de cada item como cópia do momento da compra, uma agregação sobre os pedidos resolve; caso contrário, a aplicação precisa buscar e cruzar documentos em várias idas ao banco.

No CouchDB, as consultas Mango filtram e ordenam, mas não somam: totais por cliente exigem visões MapReduce ou exportação dos dados (APACHE SOFTWARE FOUNDATION, 2026). A desnormalização acelera as perguntas previstas e dificulta as imprevistas, e por isso relatórios analíticos costumam ser servidos por um banco relacional ou por um data warehouse alimentado a partir do sistema transacional (KLEPPMANN, 2017).

7 QUESTÃO 6 — DUPLICAÇÃO DE DADOS NO PEDIDO

Em um pedido NoSQL, o nome e o preço do produto podem ser armazenados dentro do próprio documento de pedido, mesmo que esses dados também existam no catálogo. Por que essa duplicação pode ser intencional? Compare-a com a utilização de chaves estrangeiras no projeto SQL e discuta os riscos de inconsistência.

A duplicação é intencional quando representa um fato histórico, e não uma cópia a ser sincronizada. O preço de um item é o preço praticado no momento da venda: se o catálogo mudar amanhã, o pedido antigo não pode mudar junto, e o mesmo vale para o nome exibido no comprovante. Guardar esses valores no documento torna o pedido autossuficiente — legível e correto mesmo que o produto seja alterado ou retirado de linha — e dispensa leituras adicionais ao exibir o histórico (SADALAGE; FOWLER, 2012).

O contraste com o SQL é menor do que parece. A chave estrangeira garante que o item aponte para um produto existente, mas não preserva o valor da época: por isso o projeto SQL do grupo já congelava `preco_unitario` em `itens_pedido`, enquanto o projeto NoSQL copia nome e preço e ainda impede, na `validate_doc_update`, que os itens de um pedido gravado sejam alterados (FURLAN; BIRUEL; PALORCA, 2026). O risco de inconsistência surge em dois casos: quando o dado duplicado deveria acompanhar a origem, como a correção do nome de um cliente copiado em milhares de pedidos, e quando não há chave estrangeira, já que nada no CouchDB impede um item que referencia um produto inexistente. A regra prática é duplicar apenas o que deve permanecer congelado e referenciar o que precisa refletir o estado atual.

8 QUESTÃO 7 — MESMO HARDWARE, COMPORTAMENTOS DIFERENTES

Suponha que os dois projetos sejam implantados na AWS utilizando infraestrutura semelhante — CPU, RAM, rede e SSD/NVMe. Explique por que o comportamento dos sistemas pode ser diferente mesmo utilizando o mesmo hardware. Relacione sua resposta ao SGBD, índices, WAL/MVCC, LSM-tree, hashing, replicação e particionamento.

O hardware define limites; o comportamento é definido pela forma como cada SGBD converte leituras e escritas em operações de memória, disco e rede. O PostgreSQL registra cada alteração no log de escrita antecipada (WAL) antes de aplicá-la às páginas de dados e usa MVCC para que leituras não bloqueiem escritas; a latência de cada commit depende da sincronização do WAL com o disco, e as versões antigas das linhas precisam ser removidas pelo VACUUM. Seus índices B-tree atualizam páginas no lugar, com acesso aleatório ao disco (KLEPPMANN, 2017). O CouchDB também usa MVCC, mas grava em arquivos append-only: cada revisão é acrescentada ao fim do arquivo, o que torna a escrita sequencial e exige compactação periódica para recuperar espaço.

Bancos baseados em LSM-tree, como Cassandra e ScyllaDB, acumulam escritas em memória e as descarregam em arquivos ordenados e imutáveis, favorecendo altas taxas de escrita ao custo de compactações que consomem CPU e disco e de leituras que podem consultar vários arquivos (O'NEIL et al., 1996). O hashing da chave de partição distribui os dados de maneira uniforme entre nós, mas encarece consultas por intervalo (DECANDIA et al., 2007). A replicação síncrona acrescenta idas e voltas de rede a cada escrita, enquanto a assíncrona reduz a latência e aceita atraso nas réplicas; o particionamento transforma consultas que cruzam partições em operações distribuídas. Assim, com a mesma CPU, RAM e NVMe, um sistema pode ser limitado pela sincronização do log, outro pela compactação e outro pela rede entre réplicas.

9 QUESTÃO 8 — DISPONIBILIDADE E TOLERÂNCIA A FALHAS

Se um servidor do banco ficar indisponível durante uma compra, o que deve acontecer com o pedido? Compare como arquiteturas SQL com primary/replica e arquiteturas NoSQL distribuídas poderiam responder à falha. Quais compromissos existem entre consistência, disponibilidade e tolerância a particionamentos?

O pedido nunca deve ficar em estado ambíguo: ou foi confirmado com toda a baixa de estoque, ou não vale, e o cliente precisa saber qual dos dois ocorreu. Uma chave de idempotência associada à compra permite repetir a operação sem duplicar o pedido. No projeto SQL, a transação interrompida antes do COMMIT é desfeita pelo próprio banco; no projeto NoSQL, a saga grava o pedido como PENDENTE, compensa o que foi reservado ao detectar a falha e deixa a uma rotina de reconciliação os casos interrompidos no meio (FURLAN; BIRUEL; PALORCA, 2026). Em uma arquitetura SQL com primário e réplicas, a falha do primário leva à promoção de uma réplica: com replicação síncrona, nenhuma transação confirmada se perde, mas cada escrita espera a réplica; com replicação assíncrona, a escrita é mais rápida e a promoção pode perder as últimas transações. Bancos NoSQL distribuídos, como Cassandra e DynamoDB, replicam cada partição em vários nós e aceitam escritas enquanto houver quórum, mantendo a loja disponível com nós fora do ar, ao custo de réplicas temporariamente divergentes (DECANDIA et al., 2007).

O teorema CAP formaliza o compromisso: diante de uma partição de rede, um sistema distribuído precisa escolher entre responder com possível inconsistência ou recusar a operação para preservar a consistência (GILBERT; LYNCH, 2002). Abadi (2012) acrescenta que, mesmo sem falhas, há uma escolha permanente entre latência e consistência. Para pedidos e pagamentos, a consistência costuma prevalecer — é preferível recusar a compra a vender o que não existe —, enquanto catálogo, carrinho e recomendações toleram disponibilidade com consistência eventual.

10 QUESTÃO 9 — PERSISTÊNCIA POLIGLOTA

Em vez de escolher exclusivamente SQL ou NoSQL, proponha uma arquitetura de e-commerce que utilize os dois modelos. Indique qual tecnologia seria utilizada para pedidos, pagamentos, catálogo, carrinho, sessão, busca, logs e analytics, justificando cada decisão. Essa arquitetura seria mais eficiente ou apenas mais complexa?

A persistência poliglota atribui a cada componente o armazenamento que melhor atende ao seu padrão de acesso (SADALAGE; FOWLER, 2012). Em um e-commerce, pedidos e pagamentos exigem transações e integridade, enquanto catálogo, sessão, busca e eventos têm necessidades de flexibilidade, latência e volume muito diferentes. O Quadro 2 apresenta a proposta.

Quadro 2 — Arquitetura poliglota proposta para o e-commerce

Componente	Tecnologia	Justificativa
Pedidos	PostgreSQL	Transação com baixa de estoque, restrições e integridade referencial
Pagamentos	PostgreSQL	Consistência forte, trilha de auditoria e conciliação financeira
Catálogo	MongoDB ou CouchDB	Atributos variáveis por categoria e leitura do produto em um único documento
Carrinho	Redis	Escrita frequente, baixa latência e expiração automática
Sessão	Redis	Leitura por chave em poucos milissegundos, com tempo de vida definido
Busca	OpenSearch	Índice invertido, relevância e filtros por atributo
Logs	OpenSearch e armazenamento de objetos	Grande volume de escrita, consulta textual recente e retenção barata do histórico
Analytics	Data warehouse colunar	Agregações sobre grandes volumes, isoladas do banco transacional e alimentadas por CDC

Fonte: elaborado pelos autores com base em Sadalage e Fowler (2012) e em Kleppmann (2017).

A arquitetura é mais eficiente para cada carga — a busca deixa de sobrecarregar o banco transacional, o carrinho não compete com os pedidos e os relatórios não disputam o primário —, mas é inegavelmente mais complexa. Cada tecnologia acrescenta implantação, monitoramento, backup, segurança e competência específica na equipe, e a sincronização entre os bancos, por eventos ou captura de mudanças (CDC), introduz consistência eventual entre o pedido confirmado e as projeções de busca e analytics (KLEPPMANN, 2017). O ganho

compensa quando cada componente tem escala ou requisito que o banco principal não atende; antes disso, é apenas complexidade antecipada.

11 QUESTÃO 10 — DECISÃO ARQUITETURAL PARA 10 MILHÕES DE USUÁRIOS

Se você fosse responsável por projetar hoje um novo e-commerce com previsão de crescimento de 10 mil para 10 milhões de usuários, escolheria uma arquitetura predominantemente SQL, predominantemente NoSQL ou híbrida/poliglota? Justifique considerando integridade dos dados, flexibilidade, desempenho, latência, elasticidade, custo em cloud, disponibilidade, governança e capacidade operacional da equipe.

A escolha recomendada é uma arquitetura híbrida, adotada de forma progressiva e com núcleo predominantemente SQL. Com 10 mil usuários, um PostgreSQL gerenciado atende pedidos, pagamentos, clientes e catálogo — este com os atributos variáveis em colunas JSONB —, acompanhado de um cache Redis para sessão e carrinho. Essa base garante integridade transacional desde o primeiro pedido, mantém baixo o custo em nuvem, oferece governança apoiada em um esquema explícito e exige de uma equipe pequena o domínio de poucas tecnologias.

O crescimento orienta as etapas seguintes, cada uma disparada por um sinal medido e não por antecipação: réplicas de leitura e um mecanismo de busca quando catálogo e pesquisa dominarem a carga; um banco de documentos para o catálogo quando a variedade de atributos tornar o esquema um gargalo; um armazenamento distribuído para eventos e logs quando o volume de escrita exceder um servidor; e um data warehouse para os relatórios. Com 10 milhões de usuários, elasticidade e disponibilidade vêm de serviços gerenciados replicados em várias zonas, enquanto pedidos e pagamentos permanecem sob consistência forte. A decisão, portanto, não é entre SQL e NoSQL, mas sobre quando cada modelo passa a se justificar e se a equipe tem capacidade operacional para mantê-lo (KLEPPMANN, 2017; SADALAGE; FOWLER, 2012).

12 CONSIDERAÇÕES FINAIS

As dez questões mostram que a comparação entre SQL e NoSQL não se resolve pela tecnologia, mas pela distribuição de responsabilidades. O modelo relacional concentra no banco a integridade, a atomicidade e o esquema explícito, e cobra por isso em flexibilidade e em escala horizontal; o modelo de documentos entrega agregados legíveis, formato flexível e distribuição natural, e transfere para a aplicação e para a operação as garantias que o banco deixou de oferecer. A experiência das duas versões da Terra & Terra confirma esse balanço na prática: o mesmo checkout que no PostgreSQL era uma transação tornou-se, no CouchDB, uma saga com compensação, reconciliação e testes para cada falha possível.

Disso decorre o critério adotado ao longo do trabalho: modelar a partir do padrão de acesso, manter sob consistência forte o que envolve dinheiro e estoque, aceitar consistência eventual onde o negócio a tolera e introduzir cada nova tecnologia apenas quando um requisito medido a justificar.

REFERÊNCIAS

- ABADI, Daniel. Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *Computer*, [s. l.], v. 45, n. 2, p. 37-42, 2012.
- APACHE SOFTWARE FOUNDATION. Apache CouchDB documentation. Disponível em: <https://docs.couchdb.org/>. Acesso em: 16 set. 2026.
- DECANDIA, Giuseppe et al. Dynamo: Amazon's highly available key-value store. In: *ACM SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES*, 21., 2007, Stevenson. Proceedings [...]. New York: ACM, 2007. p. 205-220.
- ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de banco de dados*. 6. ed. São Paulo: Pearson Addison Wesley, 2011.
- FURLAN, Felipe; BIRUEL, Henrique; PALORCA, Henrique. *Torra & Terra: e-commerce de café especial nas versões relacional e NoSQL*. GitHub, 2026. Disponível em: <https://github.com/lipefurlan/projeto-nivaldo-nosql>. Acesso em: 16 set. 2026.
- GARCIA-MOLINA, Hector; SALEM, Kenneth. Sagas. In: *ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA*, 1987, São Francisco. Proceedings [...]. New York: ACM, 1987. p. 249-259.
- GILBERT, Seth; LYNCH, Nancy. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, New York, v. 33, n. 2, p. 51-59, 2002.
- KLEPPMANN, Martin. *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. Sebastopol: O'Reilly Media, 2017.
- MARCUSSO, Nivaldo T. *Questões para o projeto SQL e NoSQL*. Material didático da disciplina Tratamento e Armazenamento de Informação. Campinas: FACAMP, 2026.
- O'NEIL, Patrick et al. The log-structured merge-tree (LSM-tree). *Acta Informatica*, [s. l.], v. 33, n. 4, p. 351-385, 1996.
- SADALAGE, Pramod J.; FOWLER, Martin. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Upper Saddle River: Addison-Wesley, 2012.

APÊNDICE A — PROMPT UTILIZADO

Reproduz-se a seguir, na íntegra, o prompt utilizado na elaboração deste trabalho com auxílio de inteligência artificial (Claude Code, com o modelo Claude Opus 5, da Anthropic), conforme orientação do professor da disciplina. Junto com a mensagem foram anexados três arquivos: a lista de questões (Questões Projeto Sql e No-SQL.pdf) e duas atividades anteriores do grupo, usadas como referência de formato (Atividade_3.2Grupo3.pdf e Atividade_4Grupo3.pdf).

PROMPT — QUESTÕES DISSERTATIVAS DO PROJETO SQL E NOSQL (ENTREGA ABNT)

CONTEXTO

Você é um assistente acadêmico especializado em bancos de dados relacionais e não relacionais. Estou cursando a disciplina Tratamento e Armazenamento da Informação na FACAMP (2026), com o Prof. Me. Nivaldo T. Marcusso. Ao longo da disciplina, o grupo construiu o mesmo e-commerce de café especial, a Torra & Terra, em duas versões: a primeira com PostgreSQL (tabelas normalizadas, restrições CHECK e checkout transacional com SELECT ... FOR UPDATE) e a segunda com Apache CouchDB (documentos com itens embutidos, validate_doc_update, controle otimista por _rev e checkout como saga com compensação). Preciso entregar as 10 questões dissertativas do documento “Questões para o Projeto SQL e No-SQL” (anexo), respondidas e formatadas no padrão ABNT, no mesmo modelo das Atividades 3.2 e 4 do grupo (anexas).

TAREFA

Responda, de forma dissertativa e fundamentada, às 10 questões abaixo e monte o documento final formatado em ABNT.

1. Modelagem dos dados: quando a normalização do SQL é mais vantajosa e quando a agregação do NoSQL simplifica a aplicação?
2. Consistência versus flexibilidade: como cada projeto trata uma compra feita enquanto o estoque é atualizado, e qual o impacto para o cliente e para a empresa?
3. Evolução do catálogo: como produtos com atributos muito diferentes afetam os modelos SQL e NoSQL, a manutenção e a governança?
4. Escalabilidade: como escalar cada projeto em um pico de dez vezes no volume, considerando escala vertical, réplicas, particionamento, sharding e distribuição horizontal?

5. Consultas e relatórios: qual a dificuldade e a eficiência de responder, em cada projeto, a uma pergunta comercial com filtros por cidade, categoria, período e valor gasto?
6. Duplicação de dados: por que copiar nome e preço do produto no pedido pode ser intencional, em comparação com chaves estrangeiras, e quais os riscos de inconsistência?
7. Infraestrutura em cloud: por que os dois projetos se comportam de forma diferente no mesmo hardware, considerando SGBD, índices, WAL/MVCC, LSM-tree, hashing, replicação e particionamento?
8. Disponibilidade e tolerância a falhas: o que deve acontecer com o pedido se o banco cair, comparando primary/replica e NoSQL distribuído à luz do teorema CAP?
9. Persistência poliglota: qual arquitetura com SQL e NoSQL atenderia pedidos, pagamentos, catálogo, carrinho, sessão, busca, logs e analytics, e ela seria mais eficiente ou apenas mais complexa?
10. Decisão arquitetural: SQL, NoSQL ou híbrida para um e-commerce que cresce de 10 mil para 10 milhões de usuários?

COMO RESPONDER

- Extensão: 2 parágrafos por questão (aproximadamente 150 a 280 palavras cada, fora os quadros), um pouco mais concisa que nas atividades anteriores.
- Estrutura de cada resposta: (a) conceito central; (b) comparação entre o projeto SQL e o projeto NoSQL, com trade-offs; (c) exemplo concreto, sempre que possível tirado das duas versões da Terra & Terra.
- Linguagem: impessoal, acadêmica, em português do Brasil, sem bullet points dentro das respostas (texto corrido dissertativo). Quadros são permitidos apenas nas questões 4 e 9, sempre acompanhados de texto explicativo.
- Consistência com a disciplina: normalização e agregados, transação e SELECT ... FOR UPDATE, controle otimista por _rev e HTTP 409, saga e compensação, snapshot histórico, validate_doc_update, escala vertical e horizontal, réplicas, sharding e hotspots, WAL/MVCC e LSM-tree, teorema CAP e persistência poliglota.
- Nada de invenção: não citar fontes, autores, páginas ou números sem certeza de que existem, nem atribuir aos projetos do grupo algo que eles não fazem. Se um dado for aproximado, indicar isso no texto.

REFERÊNCIAS

A lista de questões, o repositório do projeto e a bibliografia abaixo, citada ao longo do texto no formato autor-data ABNT (AUTOR, ano):

- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados.
- KLEPPMANN, Martin. Designing Data-Intensive Applications.
- SADALAGE, Pramod J.; FOWLER, Martin. NoSQL Distilled.
- DECANDIA, Giuseppe et al. Dynamo: Amazon's Highly Available Key-value Store.
- GILBERT, Seth; LYNCH, Nancy. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services.
- ABADI, Daniel. Consistency Tradeoffs in Modern Distributed Database System Design.
- O'NEIL, Patrick et al. The Log-Structured Merge-Tree (LSM-Tree).
- GARCIA-MOLINA, Hector; SALEM, Kenneth. Sagas.
- Documentação oficial do Apache CouchDB e repositório do projeto (github.com/lipefurlan/projeto-nivaldo-nosql).

Usar no mínimo 4 fontes distintas ao longo do trabalho e listar todas — e apenas as efetivamente citadas — na seção de Referências.

FORMATAÇÃO ABNT DO DOCUMENTO

- Capa: FACAMP — Faculdades de Campinas; curso Engenharia da Computação; disciplina "Tratamento e Armazenamento de Informação"; título "Atividade 5 — Questões Dissertativas: Projeto SQL e NoSQL"; grupo: Felipe Furlan RA 202210339, Henrique Biruel RA 202210347, Henrique Palorca RA 202210348; Campinas; Setembro de 2026.
- Fonte Arial 12, espaçamento entre linhas 1,5, texto justificado, recuo de primeira linha de 1,25 cm.
- Margens: 3 cm superior e esquerda; 2 cm inferior e direita.
- Numeração de páginas no canto superior direito.

- Seções numeradas: cada questão vira uma seção (1 INTRODUÇÃO, 2 QUESTÃO 1 — título curto do tema, ..., 12 CONSIDERAÇÕES FINAIS, REFERÊNCIAS).
- Enunciado completo de cada questão reproduzido em itálico logo abaixo do título da seção, antes da resposta.
- Citações diretas curtas entre aspas com (AUTOR, ano, p. X); citações longas (mais de 3 linhas) com recuo de 4 cm, fonte 10, espaçamento simples, sem aspas.
- Quadros numerados sequencialmente, com título acima e indicação de fonte abaixo.
- Referências ao final, em ordem alfabética, alinhadas à esquerda, espaçamento simples e separadas por uma linha em branco.
- Incluir uma Introdução curta (1 parágrafo) e Considerações finais (1 a 2 parágrafos amarrando a ideia de que escolher entre SQL e NoSQL é distribuir responsabilidades entre o banco, a aplicação e a operação).
- Ao final do documento, acrescentar um Apêndice A — Prompt utilizado, reproduzindo integralmente este prompt (exigência do professor).

ENTREGA

Gerar o resultado em .docx e em PDF, no padrão ABNT, com o nome de arquivo Atividade_5Grupo3. Ao terminar, informar onde os arquivos foram salvos e um resumo de uma linha do que foi produzido.